Arithmetic and decimal output costs in a home computer calculation of the ten billionth Fibonacci number

Xudong Lu
Fengjing High School Affiliated to East China Normal University, Shanghai, China
Corresponding author: Xudong Lu
Email: lusunrise@126.com

Abstract

Background: Very large integers require an output and verification workflow as well as efficient arithmetic.
Methods: New experiments used an Intel Core i5-14400F host, Windows 11 and WSL2 Ubuntu. Four arithmetic implementations were measured five times at each of four index sizes. Full outputs, process-resource records and native Windows failure logs were retained.
Results: Three SHA-256-distinct Windows GMP 6.3.0 DLLs, including MSYS2, showed adjacent sparse-square outcomes at 2,147,352,576 bits (success) and 2,147,352,577 bits (wrapped allocation request). Unequal-input multiplication also failed; matched-result tests distinguished input limb lengths from mathematical result width. Three WSL2 and one MPIR target outputs matched the new 2,089,876,403-digit direct Linux reference by SHA-256.
Conclusions: The three tested DLL builds share an observed sparse-square boundary, while unequal-input tests show that mathematical result width alone does not determine failure. WSL2/Linux GMP and Windows MPIR completed the target output. The exact source expression and broader platform behavior remain untested.

Keywords: arbitrary precision arithmetic; Fibonacci number; decimal conversion; empirical evaluation; reproducible research

Introduction

An isolated large integer is not fully delivered when arithmetic finishes. Applications must also construct its external representation and preserve a complete result that another implementation or reference can check. For the ten billionth Fibonacci number, decimal representation requires more than two billion characters. This size makes the integer-to-text path a first-order engineering concern.

Efficient arithmetic methods are already available. Multiple-precision arithmetic covers size-dependent multiplication, division, and conversion strategies (Brent and Zimmermann, 2010). GMP documents the binary-prefix two-square Fibonacci algorithm used in this project (The GNU MP developers, 2023a) and employs distinct multiplication regimes (The GNU MP developers, 2023b). Improving an application at this scale therefore involves selecting and integrating existing library operations as well as managing temporary allocations and output construction.

The examined open-source program exposes several arithmetic backends, a decimal writer, and an interface that keeps diagnostics after worker failure (Lu, 2026). Its detailed narrative describes unsuccessful implementations and a completed target-size calculation. This manuscript reorganizes that account as a systems investigation, while keeping claims within the available evidence. It does not claim a new recurrence or a computational record.

Four research questions guide the analysis. RQ1 asks how the recorded arithmetic implementations compare on the available size range. RQ2 asks how much of the completed target calculation is attributable to arithmetic and to conversion with output. RQ3 asks which parts of the failure and result-validation claims can be supported by retained records. RQ4 examines full-output verification and reference provenance; RQ3 includes native-backend completion evidence. These questions are evaluated using the retained experimental evidence.

The public contribution is the complete, inspectable experiment rather than the recurrence alone: a 32 GB home computer produced and wrote all 2,089,876,403 digits, and newly generated

outputs are compared in full with direct native GMP evaluation; older artifacts are audited separately as historical material. The project preserves the path from naive recursion, int64 overflow and matrix experiments through inter-process serialization costs, algorithm comparisons, Windows allocation failure, WSL2 completion, and native MPIR completion. Its Chinese narrative exposes why an attempt failed and how the next attempt changed. Together with source, diagnostic records, and full-output checks, that record is more useful for reproduction than an isolated runnable script.

The project began with naive recursion and matrix exponentiation, followed by fast doubling, Lucas and two-square algorithms. Despite repeated algorithmic improvements, attempts to compute and write the ten-billionth term failed for an extended period. The author's exploratory GitHub search did not locate a directly usable same-scale output record; its queries and date were not retained, so no priority claim follows. Layer diagnostics eventually placed the native Windows failure at a large-integer square. The error appeared to request about 4 GiB, but a full-width allocator trace later showed a request of 2^{64} minus 369,632 bytes. An isolated square and a direct call to the same GMP DLL reproduced the failure without Fibonacci recurrence, decimal output or the gmpy2 binding. The target file was completed through WSL2/Linux GMP and native Windows MPIR. An earlier GMP development report documented an internal size overflow in large-integer multiplication (Sofroniou, 2013); its source-level cause is not assigned to the present binary.

Materials and methods

Evidence sources and scope

Experiments were run on 15 October 2026 on an Intel Core i5-14400F host with 10 physical cores, 16 logical processors and 34,072,674,304 bytes of visible physical memory. Windows reported build 27766. The failing native path used Python 3.13.7 and gmpy2 2.3.1 with GMP 6.3.0; a downloaded PyPI wheel (SHA-256 74235fcce8a1bee207bf8d43955cb04563f71ba8231a3bbafc6dd7869503d05c) supplied gmpy2 2.2.1 for two further failure trials. Its bundled `libgmp-10.dll` has SHA-256 `fb2d312ca861c18e8b23648ac8440b0b613b1930ceff58f48b48ba2d3bd6b186`, distinct from the 2.3.1 DLL. Both report GMP 6.3.0; compiler and build flags are not established by wheel metadata. WSL2 Ubuntu used Python 3.12.3, gmpy2 2.1.5 and GMP 6.3.0, with a 5.15.167.4 WSL2 kernel. Its visible memory was 16,240,976 KiB and swap 4 GiB. These are distinct runtime builds, not an operating-system-only controlled comparison. Raw environment records accompany every Linux trial.

The archived Python v4.3 source has SHA-256 f40028d222e33178aff8fd27714c4642bb2510d8cd23e1916587f948b7ee108b. It matches the pinned Git source after LF/CRLF normalization. The arithmetic and decimal-writer functions were imported without changing their formulas. A separate driver supplied native GMP, conventional fast doubling, Lucas and two-square runs. Each algorithm was measured five times at $n=10^7$, 10^8, 5×10^8 and 10^9, in fresh processes with a cyclically rotated order. All benchmark processes ran sequentially; normal host background activity was not eliminated. Arithmetic timing disables the application diagnostics callback (diag=None); it excludes module import and process startup. Timed results therefore do not include the original worker's per-layer diagnostic

logging. Main tables report medians; supplementary arithmetic_summary.csv supplies inclusive linearly interpolated quartiles and ranges; no significance test is inferred from five observations. Three WSL2 two-square trials at $n=8\times10^9$ tested the former failure scale without decimal output. Three complete $n=10^{10}$ WSL2 two-square runs were recorded, with one complete native Windows MPIR v1.1.0 release run following the first Linux trial. A final Linux native gmpy2.fib call generated a new reference at $n=10^{10}$, bypassing the application's Windows-derived dispatch cutoff. The MPIR executable is identified by SHA-256 bda33f884b98676548124096be8e747a336330355e6c8a34acaf882cd9f50921; its bytes were tested directly. Current C source, an older C snapshot and the release executable are distinguished rather than assumed to be one rebuilt object.

Linux perf_counter intervals separate arithmetic from decimal conversion and file close. The instrumented driver writes an ASCII digit-only payload using the original writer. An explicit fsync interval is recorded afterwards and excluded from conversion time; verification is timed separately. Linux resource.getrusage and /usr/bin/time -v record kernel cumulative process peak RSS. The arithmetic checkpoint and full-run peak are cumulative high-water values, not disjoint per-stage peaks. Native MPIR is timed externally from process creation to termination; its stdout stage timings have one-second granularity and are not substituted for the precise Linux intervals. Windows memory is sampled every 50 ms and includes the operating system's peak-working-set and peak-pagefile counters up to the last successful query. A final short-lived peak after that query cannot be excluded. Working set, RSS and private commitment are reported under their own names.

Each Linux result is checked modulo 1,000,000,007, 1,000,000,009, 2,147,483,647 and 10^{100} by a bounded-integer fast-doubling implementation. These checks test arithmetic consistency but do not prove every digit correct. The new native reference was streamed once more: decimal digit sums gave mod 9 = 6 and mod 11 = 0, matching bounded Fibonacci residues. Cassini's identity held modulo 1,000,000,007, 1,000,000,009 and 2,147,483,647; a period-based check gave last five digits 46875 and a separate 90-digit logarithm gave first five digits 14135. The other four new payloads share the reference's SHA-256. Each complete output is counted, checked for non-digits and hashed in bounded chunks. Primary byte comparisons use the new native GMP file and the four new repair-path outputs, removing documented headers and ASCII whitespace. Retained 2024 artifacts are compared separately for the historical audit. Full agreement is a stronger artifact check than endpoints alone; shared algorithms and libraries still limit implementation independence.

Two-square arithmetic and backend selection

The implementation maintains two neighboring Fibonacci values with their prefix index and its parity. An incoming binary digit doubles the prefix index and optionally adds one. The next state follows the established two-square identities (The GNU MP developers, 2023a):

$$F(2k+1) = 4F(k)^2 - F(k-1)^2 + 2(-1)^k$$
$$F(2k-1) = F(k)^2 + F(k-1)^2$$
$$F(2k) = F(2k+1) - F(2k-1)$$

The loop releases obsolete state references after forming the successor. For $n=10^{10}$, a 34-bit index gives 33 iterations after initialization. This is logarithmic recurrence depth, not a constant bit-operation count. The cost of each step depends on growing operands and the underlying multiplication implementation.

The program's automatic policy selects native gmpy2.fib for indices no greater than 4,294,967,294 and the Python two-square path above that value. GMP's native Fibonacci interface uses unsigned long (The GNU MP developers, 2023d). The program keeps its conservative cutoff on Linux; the supplementary driver bypasses it to compare native GMP at the same target index. The cutoff and the later allocation failure are distinct issues.

Decimal conversion and output construction

The writer plans a decimal width using a 256-bit MPFR evaluation (Fousse et al., 2007) (The MPFR developers, 2025):

$$D_n = \lfloor n \log_{10} \varphi - \log_{10} \sqrt{5} \rfloor + 1$$

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

Explicit floor is used, and the two initial sequence values are handled separately. At $n=10^{10}$, 256-bit and 1024-bit MPFR calculations have the same integer part, 2,089,876,402; the latter has fractional part 0.1503023355..., and their numerical difference is about 1.11e-68. The retained payload has 2,089,876,403 digits. These are numerical checks, not a formal interval certificate. gmpy2's documented possible one-digit excess in num_digits motivates avoiding an unchecked approximate width for padding (The gmpy2 contributors, n.d.). Fixed precision does not constitute a proof that every arbitrary index has a certified width.

The integer is recursively split into high and low decimal blocks:

$$v = q10^h + r, \; h = \lfloor \frac{w}{2} \rfloor, \; 0 \leq r < 10^h$$

The high block is written before the low block, and the low block retains its allocated width through zero padding. At widths no larger than 4,000,000 digits, a leaf is converted through gmpy2 and immediately written. Cached powers of ten are local to the task and are cleared afterwards. This follows a divide-and-conquer organization related to documented multiple-precision radix conversion (The GNU MP developers, 2023e).

The target split tree has 1,024 leaves. A complete multi-gigabyte string is avoided, but the output routine does not have memory consumption independent of n. The caller retains the full integer while recursive pending blocks, powers, and division temporaries remain allocated. The leaf threshold bounds an individual string allocation and should not be interpreted as a total-RAM bound.

Process isolation and recorded failures

The interface starts an independent worker through Python's subprocess interface (Python Software Foundation, n.d.-a). Arithmetic stages and progress are exchanged as structured messages. Diagnostic lines identify the operation about to run, the operation completed, operand lengths, and resource values when available. Each line is flushed and fsynced. GMP's default memory behavior can terminate a process on allocation failure (The GNU MP developers, 2023c), making process separation useful for preserving the interface and diagnostic evidence. Windows's LLP64 model keeps long at 32 bits even with 64-bit pointers (Microsoft, n.d.-a); this type fact alone does not identify the failed expression. The near-4-GiB message alone cannot reveal the full request. Here the allocator callback records the 64-bit value, and a direct GMP DLL call reproduces the boundary without gmpy2. This identifies a wrapped allocation size in the examined build, while leaving its exact source expression unresolved.

[Figure 1 about here]

Verification and measurement endpoints

The detailed verification protocol and raw comparison results are reported below; the measurement endpoints follow the preceding experimental protocol.

Results

RQ1 Repeated arithmetic comparisons

At $n=10^9$ the median times were native GMP 3.844 s, two-square 4.610 s, Lucas 5.622 s and conventional doubling 7.844 s.
[Table 1 about here]
Note: Five fresh-process trials per algorithm and scale; only medians appear here. Linearly interpolated Q1, Q3, IQR and ranges are supplied in arithmetic_summary.csv.
[Figure 2 about here]

RQ2 Arithmetic and decimal output costs

For $n=10^{10}$, WSL2 two-square arithmetic had median 56.95 s and conversion with file close median 551.98 s; the median sum of those measured stages was 608.93 s. The median output-stage share was 90.65%. Individual trials, explicit fsync intervals and full ranges are retained in the supplementary CSV. The median cumulative full-run peak RSS was 5.10 GiB. The historical 53.6 s, 527.8 s and approximately 5 GiB estimate remain archival observations and are superseded by the new protocol for current performance claims.
One native Windows MPIR v1.1.0 full-output run completed with external process wall time 2592.93 s and observed peak working set 8.05 GiB. Its total includes the program's width correction, initialization, progress thread and file close; no durable-file-sync measurement was added to the release executable. Because this endpoint differs from the Linux stage sum, these endpoints are not presented as a controlled implementation speedup. In particular, Python/WSL2 timings are not attributed to MPIR.
[Table 2 about here]
Note: Native is one direct gmpy2.fib control in the same WSL2/Linux environment. It is excluded from the 2S-1/2/3 medians, quartiles and figure error bars. Columns are rounded independently to two decimal places; stage sums use unrounded intervals, so adding displayed components can differ by 0.01 s. Peak is cumulative RSS; fsync and verification are separate.
[Figure 3 about here]

RQ3 Failure evidence and completed replacement paths

Failure observations

The two native gmpy2 2.3.1 trials and two isolated gmpy2 2.2.1 trials failed at layer 32/32, the first square of a 2,776,967,654-bit operand, before decimal conversion. The preserved stderr contains GNU MP: Cannot allocate memory (size=4294597664). The current 2.3.1 trials exited

with code 3 rather than the historical report's 0xC0000409. No watchdog termination was involved. Median peak working set and private commitment were 1.82 and 2.46 GiB for gmpy2 2.3.1, and 1.81 and 2.46 GiB for 2.2.1, respectively; these are distinct Windows counters. In the two gmpy2 2.3.1 runs, more than 10 GiB of physical memory remained available at the last pre-failure diagnostic. Available physical memory alone does not identify the failure; the full-width allocation trace and isolated tests below resolve the nature of the requested size.

Full width allocation trace

A separate allocator trace delegated to the original GMP allocation functions and recorded full-width size_t arguments before gmpy2 initialization. Its allocation values and log are supplied separately; traced timings are excluded from performance summaries. The largest recorded request was 18,446,744,073,709,181,984 bytes. This near-limit 64-bit size_t value has low 32 bits equal to 4,294,597,664, matching the displayed stderr size. The request equals 2^{64} minus 369,632 bytes; interpreted as a signed 64-bit integer, the same bit pattern is -369,632.

Minimal reproduction and build comparison

In protected processes on 16 October 2026, sparse operands with only the highest bit set showed adjacent square outcomes at b=2,147,352,576 and b=2,147,352,577. Define $T=2^{32}-2^{18}=4,294,705,152$ bits and $L=T/64=2^{26}-2^{12}=67,104,768$ limbs. The successful mathematical square has $T-1$ bits and the failed one $T+1$ bits. This is an observed sparse-square transition, not a universal minimum for other operand shapes. The direct DLL test and separately compiled C reproducer use the following core operations; each case runs in its own guarded process.

```
mpz_t x, y;
mpz_init_set_ui(x, 1);
mpz_init(y);
mpz_mul_2exp(x, x, b - 1);
mpz_mul(y, x, x);
mpz_clear(y);
mpz_clear(x);
```

The two wheel binaries are identified as `site-packages/gmpy2.libs/libgmp-10.dll`: gmpy2 2.3.1 SHA-256 `829adcf025d22e641c6816b431fbe5b226a39b390c7205192d480151646fe9c9` and gmpy2 2.2.1 SHA-256 `fb2d312ca861c18e8b23648ac8440b0b613b1930ceff58f48b48ba2d3bd6b186`. An independent MSYS2 mingw-w64-x86_64-gmp 6.3.0-2 package (MSYS2, n.d.) supplies a third DLL, `mingw64/bin/libgmp-10.dll`, SHA-256 `37248cda33845e7c5c35a574e57da762dd0a8268f0270d179f30a33746809681`. Its verified package SHA-256, CI packager, build date and GCC 13.2.0-2 build-environment record are supplied. All three reproduced the adjacent pair. At b=2,147,352,577, the 2.3.1 DLL requested $2^{64}-523,232$ bytes; the 2.2.1 DLL requested $2^{64}-523,232$ bytes; the MSYS2 DLL requested $2^{64}-523,232$ bytes. These requests were captured by native C allocation callbacks forwarding to each DLL's original allocator.

[Table 3 about here]

The delta column is mathematical result bits minus T. Result widths in failed cases follow from the sparse inputs; no failed result object was recovered. Limbs in this table describe the normalized mathematical result, not an observed FFT workspace length.

Scope and mechanism tests

A separate C probe compiled against the matching MSYS2 package header measured sizeof(mp_limb_t)=8, sizeof(mp_size_t)=4, sizeof(mp_bitcnt_t)=4 and sizeof(unsigned long)=4 bytes. The two wheel headers were not recovered, so their typedef widths are not established by this header probe. The inspected MPIR Win64 header instead defines 64-bit mp_size_t and mp_bitcnt_t (The MPIR contributors, n.d.). Thus type widths differ across these builds. This makes 32-bit length or workspace arithmetic a testable hypothesis; a header width alone does not identify which internal operation produced the request. GMP's mpz_t capacity fields count limbs (The GNU MP developers, 2023f), and approximately 67 million limbs remain far below INT_MAX.

Sofroniou of Wolfram Research reported a GMP 5.1.2 failure on 64-bit Linux: an int multiplication of K and l overflowed before assignment to mp_size_t, then reached allocation as a huge size_t (Sofroniou, 2013). That historical mechanism motivates checking intermediate arithmetic, rather than assigning its source expression to the present DLLs.

At six sampled square widths from 2,147,352,577 through 2,147,483,648 bits, the signed interpretation of the abnormal request stayed at −523,232 bytes. At 2,776,967,654 bits it was −369,632 bytes. Both are multiples of the 8-byte limb size, corresponding arithmetically to −65,404 and −46,204 limb-sized units; this does not prove an internal variable held either negative limb count. The plateau does not support a single nonconstant affine request-versus-raw-bit formula over those samples.

Distinct-address multiplication with unequal operand lengths also failed when the mathematical result had exactly T bits. A stronger matched-result test used bit-length pairs (2,147,352,513, 2,147,352,577) and (2,147,352,512, 2,147,352,578). Both mathematical products have 4,294,705,089 bits and 67,104,768 normalized limbs; their input limb sums are respectively 67,104,769 and 67,104,768. The former failed with the wrapped request and the latter succeeded in all three DLLs. These tests rule out mathematical result width or normalized result limb count as the sole trigger in these cases, and show that a square-specific path is not required. Input limb lengths and internal padding/workspace handling are therefore the next quantities to trace. Internal FFT lengths and the first overflowing expression remain unmeasured. All Windows tests used build 27766; no stable-Windows replication has been obtained.

At the same Fibonacci index, WSL2 two-square completed all three trials, with median arithmetic time 48.18 s and median peak RSS 2.81 GiB. The native Windows MPIR release also completed the larger $n=10^{10}$ full-output task once; the instrumented MPIR build completed three arithmetic trials at each of $n=8\times10^{9}$ and $n=10^{10}$. These observations demonstrate two usable execution paths around the observed failure. They do not constitute a patch to the original GMP binary, nor establish that all Windows GMP builds fail or that all Linux builds succeed.

The retained C and Python writers both use 4,000,000-digit leaves, but C calls MPIR mpz_tdiv_qr/mpz_get_str and fwrite whereas Python calls GMP f_divmod/digits and writes through a Python file object. The MPIR release also performs width correction and status reporting. Linux output was written under WSL2 /tmp and Windows output to C:. Conversion, padding, runtime I/O and storage waits were not measured separately, so the timing gap cannot be attributed to any one of them. Retained C source is not a binary-matched rebuild of the release.

The sampling protocol was amended after the first MPIR full-output run revealed a much longer conversion stage. The target release run was preserved, while repetition was concentrated on five complete $n=10^9$ MPIR runs and three arithmetic-only trials at each of $n=8\times10^9$ and $n=10^{10}$. The compute-only executable was rebuilt from the retained C snapshot with QueryPerformanceCounter around the unchanged Fibonacci function and an exit branch before decimal conversion. Its source, build command, library hash and executable hash are supplied separately from the release. Arithmetic medians were 107.97 s and 122.22 s at the two large indices. At $n=10^{10}$, 122.22 s is about 2.15 times the WSL2 two-square median of 56.95 s; this descriptive build comparison does not isolate an operating-system effect. The single complete MPIR target run is not presented as a repeated full-output statistic. The first release run's sampler was replaced while preserving its numeric process; the protocol amendment and both memory traces are retained, and that run's wall clock uses the operating-system process creation timestamp to observed exit.

Follow-up sink controls used $n=10^9$ and three fresh-process trials per condition, with the second round reversing condition order. The unchanged MPIR release had median process wall times 197.00 s to a C: file and 199.37 s to NUL (NUL/file=1.012). The unchanged Linux writer had median conversion-and-close times 37.90 s under WSL2 /tmp and 40.30 s under /mnt/c (/mnt/c-to-/tmp=1.063); fsync remained separate. MPIR includes arithmetic, width correction and status output, so cross-backend endpoints still differ. The stored control files matched by SHA-256; NUL produced no numeric artifact. These are within-backend sink comparisons at one billion, without a target-scale decomposition of the ten-billion timing gap.

[Table 4 about here]

RQ4 Full artifacts and reference provenance

All five newly generated files contain 2,089,876,403 decimal digits. The digit-only payloads agree in SHA-256 and full byte comparison after the MPIR prefix is removed. Their common digest is shown below.

```
6fa62cdd9d65374354899719d9a58122
2c1d497492b9bcddcb859c32bc469662
```

The first five digits are 14135 and the last five 46875. The new reference is an author-generated result of the direct Linux GMP API, with its source, command, runtime versions, environment, original stdout and resource record supplied. It provides a reproducible generation chain without relying exclusively on the old external attribution. It is not a third-party certified standard or an independent-library result. Historical files are audited only for continuity with retained material and are not primary validation evidence. Their original machine, generation command and third-party provenance remain unauthenticated; details are supplied in historical_provenance.md.

[Table 5 about here]

Discussion

The repeated runs support treating representation conversion and backend behavior as experimentally measurable parts of a numerical workflow. The software combines established identities with process isolation and bounded-leaf output. Its contribution is an inspectable implementation and failure/repair record, rather than a new recurrence or a computational world record.

The study uses one host and recorded builds. Five small-scale repeats and three Linux full-scale repeats describe local variability, without establishing population-level performance or asymptotic superiority. Host background activity and file caches were not controlled through dedicated-machine isolation; the fixed rotated algorithm order was not randomized. Linux and Windows resource endpoints differ, and cumulative kernel peaks cannot locate every allocation's contribution. The separate allocator trace changes instrumentation and is excluded from untraced timing summaries.
The wrapped allocation size is established by the full-width callback and direct DLL probe; the responsible source expression has not been established by a binary-matched source build or native stack analysis. A successful replacement backend demonstrates an operational remedy, not correction of that expression. The new reference has a documented generation chain but shares GMP and established identities with one test path. Modular checks and cross-backend agreement reduce several error risks without providing formal verification. The 2024 file's original supercomputer execution and redistribution rights remain unverified, and a public archival DOI for the new data has not been assigned.

Conclusions

New experiments demonstrate a reproducible native Windows GMP failure and two completed large-integer output paths on the recorded home computer. Three WSL2 runs and one native MPIR release run each produced the complete ten-billionth Fibonacci number, with recorded runtime and memory evidence and full agreement with a newly generated native GMP reference. Decimal conversion accounts for a median 90.65% of the WSL2 stage total. In this experiment, backend selection, temporary-memory handling and output engineering determine whether an established arithmetic algorithm can deliver a complete artifact. Broader platform claims and the specific overflowing source expression require additional controlled evidence.
Full output agrees with direct library evaluation, and modular checks satisfy the tested arithmetic relations. This is consistency evidence: the Linux paths share GMP and established identities, and matching SHA-256 digests do not independently prove numerical correctness. No independent implementation or formal correctness proof is provided.

Additional information and declarations

The software remains MIT-licensed at the pinned public Git revision (Lu, 2026). Supplementary experiment drivers, raw measurements, environment files, source and binary hashes, failure diagnostics, the isolated square/direct-DLL probes of 16 October 2026, numeric verification manifests and the reference-generation command are supplied with this revision. The new full numeric outputs are retained locally and identified by file hashes; the package does not claim that a public archive or DOI already exists. Historical files retain their original provenance qualifications. Software citation follows version-specific identification (Smith et al., 2016).
Xudong Lu conceived and supervised the project, assembled the experimental record, and is responsible for reviewing the analysis and the final manuscript.
AI assistance: AI tools assisted software development, experimental orchestration and evidence processing, and manuscript drafting, translation, and editing. Their use does not establish validity of the numerical claims. The author is responsible for scientific judgment, evidence verification,

and the final submission text, and takes responsibility for all content in the manuscript; AI systems are not authors. There were no human-subject or animal experiments.

References

Brent RP, Zimmermann P. 2010. Modern Computer Arithmetic. Cambridge University Press. https://doi.org/10.1017/CBO9780511921698

Fousse L, Hanrot G, Lefèvre V, Pélissier P, Zimmermann P. 2007. MPFR: A multiple-precision binary floating-point library with correct rounding. ACM Transactions on Mathematical Software 33(2):13. https://doi.org/10.1145/1236463.1236468

Lu X. 2026. Fibonacci 10 Billion Calculator. Python v4.3, Git revision f0c5f2488432524d6edb781cbabe1bbca2a79b6d. MIT license. https://github.com/sr-lxd/fibonacci-10-billion-calculator/tree/f0c5f2488432524d6edb781cbabe1bbca2a79b6d

Microsoft. n.d.-a. Abstract data models. Windows application development documentation. Accessed 14 October 2026. https://learn.microsoft.com/en-us/windows/win32/winprog64/abstract-data-models

MSYS2. n.d. mingw-w64-x86_64-gmp 6.3.0-2 package and build metadata. Accessed 16 October 2026. https://packages.msys2.org/packages/mingw-w64-x86_64-gmp

Python Software Foundation. n.d.-a. subprocess: Subprocess management. Python 3.13 documentation. Accessed 14 October 2026. https://docs.python.org/3.13/library/subprocess.html

Smith AM, Katz DS, Niemeyer KE, FORCE11 Software Citation Working Group. 2016. Software citation principles. PeerJ Computer Science 2:e86. https://doi.org/10.7717/peerj-cs.86

Sofroniou M. 2013. mpn_mul_fft type overflow issue. GMP developers mailing list, 18 September 2013. https://gmplib.org/list-archives/gmp-devel/2013-September/003365.html

The gmpy2 contributors. n.d. Multiple-precision integers. gmpy2 documentation. Accessed 14 October 2026. https://gmpy2.readthedocs.io/en/latest/mpz.html

The GNU MP developers. 2023a. Fibonacci Numbers Algorithm. GNU MP 6.3.0 manual. https://gmplib.org/manual/Fibonacci-Numbers-Algorithm

The GNU MP developers. 2023b. FFT Multiplication. GNU MP 6.3.0 manual. https://gmplib.org/manual/FFT-Multiplication

The GNU MP developers. 2023c. Memory Management. GNU MP 6.3.0 manual. https://gmplib.org/manual/Memory-Management

The GNU MP developers. 2023d. Number Theoretic Functions. GNU MP 6.3.0 manual. https://gmplib.org/manual/Number-Theoretic-Functions

The GNU MP developers. 2023e. Binary to Radix. GNU MP 6.3.0 manual. https://gmplib.org/manual/Binary-to-Radix

The GNU MP developers. 2023f. Integer Internals. GNU MP 6.3.0 manual. https://gmplib.org/manual/Integer-Internals

The MPFR developers. 2025. GNU MPFR 4.2.2 manual. https://www.mpfr.org/mpfr-current/mpfr.html

The MPIR contributors. n.d. MPIR Windows source and Win64 definitions. Inspected revision 5e0c2061af105c151970d41c8394ce956f77e455. https://github.com/BrianGladman/mpir/blob/5e0c2061af105c151970d41c8394ce956f77e455/gmp-h.in

Figure legends

Figure 1. Measured execution and verification workflow. Trial counts distinguish two-square runs from the direct-library control; agreement checks are not an independent correctness proof.

Figure 2. Arithmetic medians with interquartile ranges, five fresh-process trials per algorithm and scale.

Figure 3. Three WSL2 two-square full-output trials at n=10^10; bars are medians and error bars the observed range. The Native control is excluded. Conversion includes close; fsync is separate.